

SWIPE WAR

DOCUMENTATION TECHNIQUE

Systeme de Deplacement Physique & Cartes Dynamiques

Developpe pour Godot 4.6.2

Auteurs : Tajani, Ayoub, Amine, Yuri

Projet d'Etudes SAE - Jeu Video Mobile

Documentation Technique : Système de Déplacement Physique

Cette documentation explique en détail le fonctionnement du système de déplacement physique (la glissade continue) et de la détection de collision par balayage (*Sweep Check*) qu'on a codés pour Swipe War sous Godot 4.6.2.

1. Modélisation du Déplacement : Du Modèle Discret au Modèle Continu

Pour rendre le gameplay beaucoup plus dynamique et agréable, on a remplacé le déplacement classique de case en case par un système de glissade physique continue basé sur le poids et l'inertie de chaque véhicule.

A. Calcul Physique de la Distance

La distance parcourue (en pixels) dépend de la force de lancement (selon la jauge au moment du relâchement) et du poids (la masse) de la catégorie de véhicule :

$$(\text{Distance}) = ((\text{Distance Max}) \times (\text{Force})) / (\text{Masse})$$

- Distance Max : Fixée à 200.0 pixels.
 - Force : Comprise entre 0.0 et 1.0 (déterminée par le mini-jeu de timing).
 - Masse (Poids) :
 - Tank (Lourd - Masse 1.8) : Glissade courte et lourde (portée max de ~111 pixels).
 - Voiture (Moyen - Masse 1.0) : Glissade standard (portée max de 200 pixels).
 - Avion (Léger - Masse 0.6) : Glissade très longue à faible friction (portée max de ~333 pixels).
-

2. Détection de Collision par Balayage (*Sweep Check*)

Comme le déplacement se fait maintenant au pixel près de façon continue (et plus sur une grille fixe), il y avait un risque que le véhicule passe à travers un mur ou un obstacle s'il se déplaçait trop vite (effet tunnel).

Pour éviter ça, on a codé un algorithme de Sweep Check pas-à-pas :

1. Balayage à pas constant : On découpe le déplacement théorique et on vérifie la trajectoire pixel par pixel, par petits pas de 2 pixels.
 2. Vérifications géométriques à chaque pas :
 - Limites du plateau : On s'arrête si le point sort de la grille ([40, 392] en X, [180, 532] en Y, avec une marge de sécurité de 20px).
 - Obstacles centraux : On s'arrête si le point entre dans la collision des blocs centraux ([172, 260] en X, [312, 400] en Y, marge de 15px).
 - Autres unités : On s'arrête si le point s'approche d'un autre véhicule à moins de 32 pixels.
 3. Résolution : Le véhicule s'arrête net à la dernière coordonnée pixel valide calculée avant l'impact.
-

3. Mini-Jeu de Timing (*Perfect Launch*)

Pour ajouter un côté adresse, le déplacement utilise une jauge de puissance qui oscille automatiquement (effet Ping-Pong) :

- Calcul de l'oscillation : La jauge de puissance oscille en continu entre 0% et 100% via une fonction basée sur le temps écoulé (delta). Un aller-retour prend exactement 1,25 seconde.
- Zone Parfaite (*Perfect Launch*) : Elle est placée stratégiquement entre 80% et 90% de la jauge.
- Ce qui se passe quand on réussit :
- Feedback Visuel : L'aiguille devient dorée et vibre légèrement à l'écran (grâce à une petite vibration sinusoïdale haute fréquence appliquée sur son orientation).
- Bonus Gameplay : Si on relâche le clic pile dans cette zone, le déplacement est gratuit (0 PA consommé), un texte "Perfect Launch!" s'affiche, et une traînée de propulsion arc-en-ciel néon s'active derrière le véhicule.

4. Dessin des Véhicules : Rendu Vectoriel Procédural

Tous nos véhicules sont dessinés mathématiquement avec la fonction `_draw()` de Godot, sans aucune image PNG externe. Pour orienter les dessins facilement selon la direction du véhicule, on utilise une matrice `Transform2D` basée sur l'angle de visée (`facing.angle()`). Dans le repère local, l'avant du véhicule pointe toujours vers la droite (Axe +X).

Voici le détail de la superposition des couches géométriques pour chaque modèle :

A. Le Tank (Modèle Blindé)

Le dessin du char d'assaut superpose 6 étapes géométriques :

1. Ombre portée : Un simple cercle noir transparent (`Color(0,0,0,0.4)`) décalé de 3 pixels vers le bas pour donner du relief.
2. Chenilles (Treads) : Deux rectangles métalliques sombres (`#1E2026`). Une boucle `for` dessine des lignes noires transversales (`#2D313B`) espacées de 4 pixels pour représenter les crampons, et 5 disques intérieurs représentent les roues de route.
3. Châssis polygonal : Un polygone à 6 sommets (`draw_colored_polygon`) formant un blindage frontal incliné (bleu acier pour l'allié, bordeaux pour l'ennemi) entouré d'une ligne de contour néon (`draw_polyline`).
4. Réacteur arrière : Un triangle orange/bleu avec un coeur blanc translucide pour simuler l'échappement de propulsion.
5. Canon Railgun : Un rectangle de métal blindé entouré de 3 lignes verticales lumineuses simulant des bobines d'induction magnétique électromagnétiques, terminé par une étincelle blanche à la bouche.
6. Tourelle centrale : Un cercle central avec son noyau lumineux (`draw_circle`) et des lignes de mire blanches en forme de croix tactique.

B. La Voiture (Modèle Intercepteur)

Conçue pour un look super profilé et rapide :

1. Ombre portée : Cercle translucide de rayon 14px sous le véhicule.
2. 4 Roues néons : Des petits rectangles noirs pour les pneus, bordés par des cercles de jantes gris, avec un segment de ligne néon active brillante sur le flanc extérieur de chaque roue.

3. Châssis aérodynamique : Un polygone aérodynamique à 7 sommets formant un avant pointu et un double spoiler (aileron) à l'arrière, souligné par deux bandes néons décoratives de course parallèles.
4. Double canons à plasma : Deux canons rectangulaires montés de part et d'autre des flancs extérieurs avec des pointes de tir lumineuses.
5. Cockpit en dôme : Un disque en dôme de verre (draw_circle) teinté bleu/rouge avec un segment blanc diagonal matérialisant le reflet de la lumière sur la vitre.

C. L'Avion (Modèle Glisseur Lévitant)

Donne un effet de lévitation magnétique :

1. Ombre portée décalée : Projetée à 6 pixels vers le bas (au lieu de 3px) et plus diffuse, ce qui donne l'impression de hauteur et de lévitation au-dessus du sol.
2. Ailes en delta : Un polygone profilé à 6 sommets formant des lignes delta agressives.
3. Feux de navigation : Deux petits cercles clignotants vert/rouge clignotants au bout des ailes (Wingtips).
4. Tuyère de postcombustion : Un grand triangle de plasma translucide à l'arrière avec un coeur blanc très dense.
5. Canon Laser de proue : Une longue pointe de tir fine s'étendant à l'avant du nez de l'appareil.
6. Cockpit en diamant : Un polygone à 4 sommets dessinant une verrière profilée en forme de diamant.

D. Squash & Stretch (Dynamisme visuel)

Lors d'un déplacement, le script applique temporairement un étirement géométrique de 1.25x sur l'axe de marche et une compression de 0.75x sur l'axe perpendiculaire. Ce décalage est géré par un Tween élastique rapide, créant un effet d'impact et de déformation très naturel à l'arrêt (l'effet de "juice").

5. Conception de la Carte et des Obstacles

L'arène de combat est un plateau quadrillé dessiné en arrière-plan à chaque rafraîchissement d'écran dans `_draw()`.

A. Dimensions de la Grille

- Structure : Il s'agit d'une grille de 8x8 cases (`grid_size = 8`).
- Résolution des Cases : Chaque case mesure exactement 44x44 pixels (`cell_width = 44`), ce qui donne une surface de jeu de 352x352 pixels.
- Centrage : Pour s'adapter à l'écran mobile tout en laissant de la place pour l'interface, la grille est décalée de 40 pixels en X (`offset_x = 40`) et de 180 pixels en Y (`offset_y = 180`).
- Technique de Tracé : Le fond est un rectangle sombre opaque (`#0F0F11` à 80% d'opacité). Les lignes de la grille sont dessinées avec une boucle simple et la méthode `draw_line` en couleur contrastée sombre (`#1D1D21` d'épaisseur 2px).

B. Cartes Dynamiques et Obstacles Génériques

Au lieu de figer l'obstacle central dans le code, le système utilise maintenant des coordonnées de cellules dynamiques. Chaque case d'obstacle est stockée sous forme de coordonnées (x, y) sur la grille de 8 x 8 cases (où x, y ∈ [0, 7]).

- Calcul de la zone d'une Case : Pour n'importe quelle case d'obstacle (cx, cy), ses limites en pixels à l'écran

sont calculées de façon générique :

- $x_{((min))} = (offset_x) + cx \times (cell_width)$
- $x_{((max))} = (offset_x) + (cx + 1) \times (cell_width)$
- $y_{((min))} = (offset_y) + cy \times (cell_width)$
- $y_{((max))} = (offset_y) + (cy + 1) \times (cell_width)$
- Esthétique par Case : Chaque case d'obstacle est dessinée individuellement dans `_draw()` avec :
 1. *Une base physique* : Un carré de taille 44x44 de couleur gris anthracite sombre (#18181A à 90% d'opacité).
 2. *Une lueur technologique* : Un carré interne rétréci de 2 pixels (40x40), de couleur bleue néon (#00D2FF à 15% d'opacité) simulant un bouclier ou un champ énergétique interactif.

6. Génération et Changement Dynamique de Cartes (Multi-Map)

Pour diversifier les parties et montrer que l'algorithme de collision est flexible, on a créé un système de presets de cartes interchangeables à la volée.

A. Les Presets de Cartes Disponibles

Les cartes sont définies par un dictionnaire global `MAP_PRESETS` qui associe un nom à une liste de coordonnées `Vector2i` :

- Classic ("classic") : Le bloc de 2x2 cases centrales classique (cases (3,3), (3,4), (4,3), (4,4)).
- Cross ("cross") : Une croix barrant le centre, créant des couloirs étroits de contournement.
- Pillars ("pillars") : Quatre piliers isolés situés aux coins centraux (cases (2,2), (2,5), (5,2), (5,5)), parfaits pour le tir à couvert.
- Corridor ("corridor") : Deux blocs horizontaux bloquant les côtés latéraux pour forcer les joueurs à s'affronter dans un goulot d'étranglement central.

B. Comment Ajouter un Nouveau Preset de Carte ?

Créer une nouvelle carte est hyper simple. Il suffit d'ajouter une ligne dans le dictionnaire `MAP_PRESETS` en haut du script `ui_test_scene.gd` :

```
const MAP_PRESETS = {
  "classic": [Vector2i(3, 3), Vector2i(3, 4), Vector2i(4, 3), Vector2i(4, 4)],
  "cross": [Vector2i(3, 2), Vector2i(3, 5), ...],
  # AJOUT DE VOTRE NOUVELLE CARTE EXEMPLE : "LABYRINTHE"
  "labyrinth": [
    Vector2i(1, 1), Vector2i(2, 1), Vector2i(5, 1), Vector2i(6, 1),
    Vector2i(3, 3), Vector2i(4, 3),
    Vector2i(1, 6), Vector2i(2, 6), Vector2i(5, 6), Vector2i(6, 6)
  ]
}
```

C. Gestion Dynamique des Collisions (*Sweep Check* Générique)

La détection de collision s'adapte automatiquement à n'importe quel preset de carte actif grâce à la fonction `_is_position_colliding_with_obstacles(check_pos, obs_margin)` :

1. Elle boucle sur toutes les cellules de la carte active `obstacle_cells`.
2. Pour chaque cellule, elle détermine ses frontières physiques en pixels.
3. Elle applique la marge de sécurité physique `obs_margin` (généralement 15.0 pixels) et retourne `true` si le véhicule ou la prévisualisation pénètre cette zone.

Grâce à cela, le tir laser, les trajectoires de l'IA, le tracé du cône holographique de visée et le déplacement physique s'adaptent instantanément et sans bug de collision.

D. Changement de Carte à la Volée

On a mis un bouton tactile "CARTE : [NOM]" sur l'interface qui permet de :

1. Parcourir circulairement la liste des presets du dictionnaire `MAP_PRESETS`.
2. Mettre à jour les cellules actives dans `obstacle_cells`.
3. Afficher une notification de texte flottant de couleur bleu cyan pour informer de la transition.
4. Forcer le rafraîchissement d'écran (`queue_redraw()`) pour redessiner la grille avec ses nouveaux obstacles, créant un effet interactif instantané.

7. Mode Sandbox pour les démonstrations

Pour que ce soit hyper simple de montrer le jeu en direct (devant le prof ou pendant une présentation), on a configuré un mode Sandbox pratique :

- IA en pause : L'IA ne joue pas ses tours toute seule pour nous laisser tout le temps d'expliquer le code tranquillement.
- Tours et PA infinis : Dès qu'on clique sur le bouton "End Turn", tous nos Points d'Action (AP) se rechargent instantanément au maximum (5 AP) pour continuer la démonstration à l'infini.
- Cibles d'entraînement immobiles : Les tanks ennemis restent sur place et servent de cibles pour montrer le tir laser, l'impact physique (secousses) et les explosions de particules.